of sync meetings at a time if you can help it.
- Do not schedule meetings during the last hour of your day (if you can help it). This leads to working late, typically there are always items that must be wrapped up by the end of the day

Diversity

- Bias toward Async communication
- Make Family feel welcome through activities such as Family and Friends Day, and Create Team Day

Iteration

- Using the Large MR Dashboards to identify lessons learned for iteration
- Large Backend Merge Requests (internal only)
- Large Frontend Merge Requests (internal only)
- Track Merge Requests that have been open for 8+ days and investigate if there are opportunities for improved iteration.

Transparency

- Dogfooding GitLab as much as possible will lead to increased transparency
- Weekly Engineering Manager Announcements are created weekly in an Issue

---
SECTION: engineering/devops/dev/create/engineering-managers/meetings.md

---
SECTION: engineering/devops/dev/create/engineering-managers/monitoring.md

How do we monitor how we are doing?

The Create Stage uses logs as a type of asynchronous retrospective. When there is a metric that we are focusing on improving, we will take the additional step to begin logging behaviors related to that metric. We look at the metric, and ask ourselves, "What is the story behind this number?"

Stories lead to trends

Looking at metrics across a team, across the stage or across an organization begins to tell a story. Each metric is a breadcrumb. As you piece the story together, if you are lucky, you will recognize trends. Once you identify trends, you can begin making changes and measuring the impact of those changes. Those changes in turn may or may not change metrics or trends. It is a big experiment and an opportunity to learn and improve!

Throughput Log

The entire Dev Sub Department makes a monthly entry into a Throughput Log. Each Engineering Manager in Dev is responsible for updating this document (internal).

The Throughput Log includes the following information:

- Team Name
- MR Rate from two months ago
- MR Rate from last month
- A percentage that reflects how much the MR rate either increased or decreased comparing the two months
- Bulleted list of explanations to describe events / activities / processes that led to the change in the MR Rate

This log gives the Dev Director visibility across the entire sub department. This log gives the Director the transparency to enable them to act if necessary in support of all of the Individual Contributors that report up through the Director.

Days to Merge Log

The Create Stage Engineering Managers makes a monthly entry into their own confidential Days to Merge Log. Each Engineering Manager is responsible for analyzing all merge request that took eight days or longer to merge and analyze these merge request to determine if there are any lessons to be learned from them.

The Days to Merge Log includes the following information:

- Team Member
- Number of Days to Merge
- Merge Request (linked title)
- Is this related to the team processes? If yes, please explain.
- Is this related to the individual team? If yes, please explain.
- Is this an exceptional MR that could not have been broken down?

This log gives Engineering Managers more transparency into how their team is working. If there are common themes month after month, these themes can serve as a guide for areas for the Engineering Manager to focus on.

The Senior Engineering Manager gains visibility across the entire stage from this log. There might be opportunities for the Senior Engineering Manager to support the Engineering Managers in their efforts.

SECTION: engineering/devops/dev/create/engineering-managers/okrs.md

Cascading OKRs

GitLab OKRs allow us to align our work to the overall GitLab objectives. Create Stage OKRs are typically derived from the Sub Department, Department and Division where it resides. Impactful OKRs outisde of this cascade are allowed to be set provided a signoff is gotten from the Stage.

1. GitLab OKRs.
1. Engineering OKRs.
1. Development OKRs.
1. Dev OKRs.
1. Create Stage OKRs.

Team OKRs

Code Review
IDE
Source Code

OKR Authors

Senior Engineering Managers, Engineering Managers and Staff Engineers are all responsible for creating OKRs.

OKR Format

All OKRs have the following format:

1. IACV
1. Product
1. Great Team

OKR Timeline

The GitLab OKRs run for one quarter (3 months). Here is a sample timeline that we follow:

Month 1

Week 1 - This week EMs are scoring OKRs from the previous quarter that may not have been scored.
Update the final OKR scores from the previous quarter
Complete the OKR Retrospective from the previous quarter
Close out the previous quarters' OKRs

Week 4
Update the current progress (scores)
Update OKRs if business priorities have recently changed or if you realize they are unrealistic
Remove OKRs if business priorities have recently changed
Add new OKRs if business priorities have recently changed

Month 2

Week 4
Updating current progress (scores)
Update OKRs if business priorities have recently changed or if you realize they are unrealistic

Remove OKRs if business priorities have recently changed
Add new OKRs if business priorities have recently changed

Month 3

Week 2 - This week EMs will begin identifying organizational KRs that are applicable to their teams including KRs they would like their team to focus on.

Begin working on the next quarter's OKRs

Week 4 - This week EMs are wrapping up the current OKRs for this quarter as well as finalizing the OKRs for the next quarter.

Begin finalizing the OKR score updates for the current quarter
Finalize the next quarter's OKRs.

Guidelines for Writing Effective Key Results

These are the best practices recommended for writing Key Results:

Create Key Results that are challenging

Completing 70% of the key result would be challenging

A good rule of thumb is to determine a goal that is definitely achievable and then change it a bit until it becomes a stretch goal

Create clear and transparent Key Results

Avoid Acronyms and jargon

Do not include day to day activities

Daily tasks such as attending weekly meetings are not valid Key Results

Create specific measurable Key Results

Key Results must include a number

Don't overwhelm yourself or your team

Create no more than three to five Key Results per objective
Create no more than nine Key Results in total for the entire quarter

Scoring OKRs

Engineering Managers score and update their OKRs on a monthly cadence. Having managers check in on a monthly cadence ensures that managers maintain focus on the quarterly OKRs.

OKRs will be scored according to this process

Reporting on OKRs

The OKRs will be reported on using Epics and Issues. The Create Stage OKRs can be viewed from this OKR Issue Board.

Good Label Hygiene

All KRs should have the following labels:

~OKR
~"Development Department"
~"Dev Sub-department"
~"devops::create"
one of: ~"group::source code", ~"group::code review", ~"group::editor", ~"group::gitaly" or ~"group::ecosystem".

OKR Retrospectives

As we close out the quarterly OKRs, the Create Stage Engineering Managers use the following OKR Retrospective format below.

RETROSPECTIVE

Good

Things that went well. If your team accomplished your goal, what contributed to the team's success?

...

Bad

Things that did not go well, but are generally one-off events·things that we don't expect to repeat. If your team didn't accomplish your goal, what obstacles were encountered?

Given the high number of OOO and the realignment changes, the MR Rate should have been reduced by at least 30%. This rate was changed during the quarter. We have to do a better job anticipating how significant OOO impacts our MR Rates.

Do Better
Things that we can do better next time. These can include a suggestion on how to do it better.

...

Best

Things that went really well. Celebrate! How can we do more of this?

...

Feels + Open Questions

Emotions, mindsets, areas of confusion, and opportunities to consider.

...

Additional Questions

Was the goal harder or easier to achieve than you'd thought when you set it?
If we were to rewrite the goal, what would we change?
What have we learned that might alter our approach for our next cycle's OKRs?

SECTION: engineering/devops/dev/create/engineering-managers/training.md

What training do we recommend for Engineering Managers?

CREDIT Values

At GitLab we see CREDIT everywhere, it is in our 360 Feedback, Annual and Mid Year Performance Reviews and in Promotion Documents.
It is important to have evidence for this CREDIT but the training below provides opportunities to learn more about how
Engineering Managers can live our CREDIT values.

GitLab Value	Online Course(s)
Collaboration	Course(s) from the Building Trust and Collaborating with Others Pathway
Results	Managing for Results
Efficiency	Efficient Time Management
Diversity	Course(s) from the Diversity, Inclusion, and Belonging for Leaders and Managers Pathway
Iteration	Interview about Iteration
Transparency	Communicating with Transparency

Working Remotely

We are experts in working remotely, so our handbook is the best resource for this area.

- Remote Work Foundations (Handbook)
- How to Manage a Remote Team

Feedback

Giving and Receiving feedback is challenging. The training below provides some best practices and ideas regarding
how to handle feedback.

- Giving & Receiving Feedback

New Managers

- New Manager Foundations
- Become a Manager
- Transitioning from IC to Manager
- Avoiding New Manager Mistakes

Recognition of Team Members

- Recognizing Team Members
- Giving & Receiving Feedback
- Delivering Employee Feedback

Coaching

- GitLab Coaching Framework (Handbook)
- Improve Your Coaching Skills as a Manager

Mentoring

Mentoring sounds easy however there can be pitfalls.
The training below is a good resource to ensuring you engage in a productive Mentoring relationship.

- Grow your Impact as a Mentor (Handbook)
- Mentoring(Handbook)

Leadership

- Emotional Intelligence(Handbook)
- Elevate Manager Training(Handbook)
- Become a Leader
- Leading Others Effectively
- Become a More Resilient Leader in Turbulent Times
- Leadership Fundamentals

High Performing Teams

- Psychological Safety: Clear Blocks to Innovation, Collaboration, and Risk-Taking

Learning GitLab

- Learning GitLab
- Continuous Delivery with GitLab

SECTION: engineering/devops/dev/create/engineers/_index.md

Who we are

Create

Create:Code Review Backend

{{< team-by-manager-slug "francoisrose" >}}

Create:Code Review Frontend

{{< team-by-manager-role role="Senior Engineering Manager(.)Create:Code Review" team=".(Frontend|Fullstack).Create:Code Review" >}}

Create:Remote Development

{{< team-by-manager-slug "adebayoa" >}}

Create:Source Code Backend

{{< team-by-manager-role role="Senior Engineering Manager(.)Create:Source Code" team=".(Backend).Create:Source Code" >}}

Create:Source Code Frontend

{{< team-by-manager-role role="Senior Engineering Manager(.)Create:Source Code" team=".(Frontend|Fullstack).Create:Source Code" >}}

SECTION: engineering/devops/dev/create/engineers/books.md

What books do we recommend for Engineers?

New Team Members

- The First 90 Days, Updated and Expanded: Proven Strategies for Getting Up to Speed Faster and Smarter

Time Management

- Make Time: How to Focus on What Matters Every Day
- The Surprising Science of Meetings: How You Can Lead Your Team to Peak Performance

Results

- The 4 Disciplines of Execution: Achieving Your Wildly Important Goals
- Getting Things Done: The Art of Stress-Free Productivity
- Atomic Habits: An Easy & Proven Way to Build Good Habits & Break Bad Ones
- The Power of Habit: Why We Do What We Do in Life and Business
- Failing Forward

Diversity Inclusion & Belonging

- Wisdom at Work
- The Culture Map: Breaking Through the Invisible Boundaries of Global Business
- The 21 Irrefutable Laws of Leadership: Follow Them and People Will Follow You

---
SECTION: engineering/devops/dev/create/engineers/conferences.md

Why is it important to attend conferences?

Attending conferences offers the following advantages:

Networking
Improve problem solving abilities
Learn new things outside of your interest
Share learnings with your team members at GitLab
Hear about the latest research

Networking

Conferences provide opportunties to network with peers using the same technologies that you are currently using or plan to use in the future. You can build relationships that will last beyond the conference days that could help you learn and grow in addition to what you learned from the conference.

Improve problem solving abilities

New topics learned from these conferences might introduce you to new approaches to solving old or current problems that you are currently facing at work. You also might have opportunties to ask others if they have had similar problems and how they have overcome them.

Learn new things outside of your interest

Surprisingly there might be something new that you are exposed to. You might identify a new area to explore!

Share learnings with your team members at GitLab

The real power in these conferences is how you use your learnings when you return from the conference. Sharing insights, solutions and ideas with the team members who were unable to attend could be very impactful to your team.

Hear about the latest research

Conferences are a great place to learn about the latest innovations. Attending these conferences will keep your skills sharp and competitive with the rest of the industry.

What are some conferences going on this year?

Conferences for Frontend Engineers

Web Directions Topic

Conferences for Backend Engineers

RedisConf
Gophercon

GraphQL

GraphQL Summit

What are some benefits to presenting at conferences?

On top of all benefits of attending the conference mentioned above, when speaking you also have a chance to

Share your knowledge and experience with the community
Bolster your personal brand and spread the word about GitLab
Improve your public speaking and communication skills

For more information on speaking at conferences, check out the Developer Advocacy CFPs handbook.

How do I go about requesting permission to attend a conference?

Contact your manager and inform them of your interest. Conference attendance will require manager approval. Once approved an issue such as this one should be created to determine if there is additional interest by other GitLab team members.

SECTION: engineering/devops/dev/create/engineers/iteration.md

How do we iterate?

We conduct Iteration Retrospectives on a monthly basis. The purpose of the Iteration Retrospective is to take a moment to reflect on how we can improve as a team on our iteration or learn lessons from a successful iteration. The lessons learned from an interaction retrospective will be an opportunity for growth for many.

All teams in the Create Stage will be conducting these retrospectives which will also facilitate cross team shared best practices and experiences.

Teams can select successful or unsuccessful examples of Iteration.

Frequency

These retrospectives should be conducted quarterly.

Before the Iteration Retrospective

Team Members / Engineering Manager should look through past issues they've worked on to find one good (challenging) candidate for the Iteration Retrospective. A retrospective issue should be created once a candidate has been chosen.

During the Iteration Retrospective

Team Members and the Engineering Manager should discuss alternative methods of breaking down the problem asynchronously in the issue. This activity should be timeboxed as the idea is to keep team members engaged for no more than a week.

Iteration Retrospective Template (optional)

This is an example of a template available for you use:

Issue Description should include:

+ Summary of the Issue
+ Did the team meet the iteration goals? Why or why not?
+ How many MRs were created to complete the Iteration
+ What were the Days to Merge for the MRs related to the MR
+ Are there successes or opportunities for improvement with respect to collaboration with peers or stable counterparts?
+ Could the original issue could have been broken down into smaller components?
+ Could the MR(s) have been broken down into smaller components?

Comment 1: Topic: What aspects of the issue / MR were very good examples of Iteration and why?

+ The team members share their positive reactions to how well iteration was applied.

Comment 2: Topic: Does anyone have alternative ideas for how this work could have been broken down

+ The team members would contribute different ideas or approaches here.

Comment 3: Topic: Is there anything that we can change in our processes or the way we work that could improve our iteration as a team?

The team members would contribute different ideas or approaches here.

SECTION: engineering/devops/dev/create/engineers/professional-development.md

How do we support Professional Development?

At GitLab we support our team members by focusing on and prioritizing their professional development. There are a variety of learning opportunities available to team members, and in some cases, community contributors. The first step in this process is for conversations to occur between the Engineering Managers and their train members to define goals, for transparency.

Promotion Process

You are the main driver in your career. You don't need to wait for your manager to tap you on the shoulder and ask if you are interested or feel like you are ready for a promotion. You are a manager of one. You are empowered!

If you feel that you have earned a promotion, take a look at the Job Family and Values for the position you are interested in. If you believe you are ready, talk to your manager and share your evidence.

Let them know that you are starting a Promotion Document, and once you are done, you would like your manager to review it. Your manager will be eager to support you by either moving your promotion document forward or telling you whether or not you need more evidence or more consistent behaviors. Either way, discussing promotion with your manager will serve you well by ensuring a plan is in place to help get you to the next level. If you know you want to work towards a promotion but are not ready yet, you can use the promotion doc templates below to help drive your employee development conversations.

GitLab's general promotion document can be found here.

Below you will find a copy of the general promotion document that includes specific examples to help guide questions to ask yourself for each pillar of the promotion document aligned with role-specific criteria:

Promotion from Senior to Staff

Goals

Define your Goals

Only you can determine what you want to do and how you want to grow professionally. The first step is to define what you want to do next. Create a 1 year plan for yourself and communicate this to your manager during your Performance Review as part of your Next Steps & Opportunities.

Track your Goals

Identify the mechanism to track your progress on your goals. The recommended approach would be to use GitLab to create issues in a personal project to track your goals. Having everything in this format will make it transparent for both you and your manager to keep track of how you are progressing with respect to your Professional Development.

Working on Challenging Tasks

Challenging tasks are hidden gems in terms of upskilling your current skill set. There are some ideas below:

1. Talk to your Manager
1. Do the hard work
1. Keep your eyes open
1. Look at mistakes as a gift or opportunity

Talk to your Manager

Ask your manager for exactly what you want. If you know the work you would like to do to expand your knowledge and grow your skills, be very transparent with your manager. If you don't know exactly what you want to do, share that also. Your manager will work with you to help you on your journey.

Do the hard work

The hard work could be highly visible or rarely seen. The visibility does not matter: the level of difficulty should be the focal point. Sometimes you may be surprised by how difficult something is. Maybe it was a difficult problem disguised as an easy fix or perhaps it was obvious from the beginning that it was difficult. Hard work requires you to collaborate with your team members, grow your iteration skills and technical expertise producing an effective result. That is three values for one specific effort. This is definitely a good option for growing skills and gaining more knowledge.

Keep your eyes open

GitLab has many communication channels. The good news about that is that all of the challenges and problems we have are all written down somewhere. Here are a few approaches you can take:

If your manager or someone in GitLab management is looking for volunteers to work on an initiative, a working group or even an Epic in the area you are interested in, volunteer!
Search through Epics, Issues and Slack to find people working on problems you are interested in.
Make it a goal to align your career development with the problems that GitLab is facing.
Attend group conversations for areas you are interested in, or follow agendas for areas you are interested in. Listen for areas GitLab is focused on and determine if any of them align with your interest.

Look at your mistakes as a gift

Each time you make a mistake, you learn something. If you have not made many mistakes in your professional development in its early stages. This is fine and perfectly normal if you are early in your career. However as you grow in your career, you grow through your mistakes and failures and you learn not to be afraid of them but to embrace them, learn from them and over time you will appreciate them.

Growing Others

Teaching someone how to do something improves your knowledge on that subject matter while also growing the skills of another. This is a win - win opportunity that should not be overlooked.

Expanding Knowledge through Reading & Online Courses

Sometimes you need to pick up a good book and just read. Maybe you would prefer an audio book? Either way, we all have limited knowledge and reading exposes us to new ideas and increases our intelligence.

Another option for this depending on the best way you learn would be online courses. GitLab has online courses they have developed. GitLab has also purchased Linked In Learning licenses and made them available to GitLab team members.

Training for Engineering Managers
Training for Engineers

GitLab Sponsored Programs

GitLab has several programs they have in place which offer additional learning opportunities.

1. Internal Internship Program
1. Shadow Programs
1. Mentoring Programs

Internal Internship Program

Internship for Learning Program
If your manager has coverage, you can spend a percentage of your time working (through an 'internship') with another team.

Shadow Programs

Security Shadow Program

From converging on real-time critical events with SIRT, exploiting vulnerabilities with the Red Team or participating in live Customer Assurance calls with the Risk and Field Security team, you will have the opportunity to work next to security staff to gain valuable insight and working knowledge of security fundamentals across multiple domains.

CEO Shadow Program

The goal of the CEO Shadow Program is to give current and future directors and senior leaders at GitLab an overview of all aspects of the company.

Chief of Staff to the CEO Shadow Program

The Chief of Staff to the CEO may occasionally have a Chief of Staff to the CEO Shadow, a GitLab team member who will participate in a specific project or initiative for a fixed period of time.

Mentoring Programs

Self Select Mentoring

If you choose, you can reach out to a team member who is more senior than you and request for them to Mentor you.
Here is a format for how to build a mentoring relationship

Minorities in Tech Mentoring Program

This program was offered last year for Underrepresented team members.

Career Changes

Sometimes team members want to develop professionally, but in a different role!

If you find that you are ready for a role change. Talk to your manager. For example, if you want to transition from a Frontend Engineer to a Backend Engineer or to a Security Engineer. Let your manager know - we want you to have the richest career possible at GitLab and we will support team members on that journey.

SECTION: engineering/devops/dev/create/engineers/skip-level.md

Why are Skip Level meetings important?

Skip Level meetings provide a wealth of opportunities

- Team members can provide the Senior Manager valuable feedback
- Senior Manager can give the team member praise for their most recent accomplishments
- Senior Managers can learn more about what is going on in their organization
- Team members get the benefit of getting coaching advice from Senior Managers

Receiving Feedback

1-1 meetings are a great opportunity for direct reports to give feedback to their Managers, however Skip Level meetings afford the same opportunities. In the case that the feedback is not given or accurately received, the Skip Level Meeting gives the team member another chance to communicate feedback. The feedback can be specific to the team members manager, team processes, GitLab policies, GitLab leadership or whatever the team member would like to share about the company. This feedback can lead to strengthening of our CREDIT values. Here are a few results that have come from some of my Skip Level Meetings

- Creation of new Handbook Pages (e.g: Transitioning from an Individual Contributor to a Manager)
- Changes to teams Sprint Planning Process
- Engineers requesting more direct feedback from their Managers

Giving Praise

Skip Level Meetings are a unique opportunity for the Senior Manager to layer on the praise! The team members here at GitLab do such an awesome job, that we can't say thank you enough. However, Senior Managers are not sometimes privy to the day to day wins that don't make it quite to the level of a Discretionary Bonus. The Senior Manager can capture praise from the following locations and share with the team member during the Skip Level Meetings:

- Make time to review Praise sections from the Individual Team Retrospectives
- Make time to review the Thanks channel looking for team members from your organization
- Check in with the team member's manager to git the latest accomplishments

Information

During Skip Level meetings, the Senior Manager can learn about the work the team members are doing and also they can learn about the health of the team from a different perspective. Typically Senior Managers learn about how things are going with the team from the Manager's perspective, so this is additional data for the Senior Manager to learn more about the health of the team. The health of the team refers to the team morale, internal processes and results.

Coaching Opportunities

Skip Level meetings give the Senior Manager an opportunity to coach team members. Each team member has a support team. Their direct manager can serve as a coach as well as their direct manager's manager. We all want to see team members succeed so if there is an opportunity for a coachable moment during the Skip Level meeting, it is welcomed.

How do we lead Skip Level meetings efficiently?

The Senior Manager meets with team members across the Create Stage in an effort to get a better sense of the work environment challenges facing the Engineers. Engineers have an opportunity to share their perspectives on everything GitLab.

This feedback will be a tool that we can use to improve communication and your work environment. For that reason, the feedback will be shared with your Manager. (If you have a topic that is sensitive in nature that you would like to discuss with me confidentially, we can reschedule a meeting specifically for that)

Audience

This is the template used by the Create Stage Senior Manager. However this template is available if anyone viewing this template finds that it would be useful for their Skip Level Meetings.

Async Meeting Prep

Senior Manager Prep

Typically the prep work for these meetings are pretty simple as all they require is scheduling and copying a default template.

Engineer Prep

Prior to the meeting, the Engineers are asked to:

- Complete the default template below filling in only what is applicable to them

1st Skip Level Template

The template below is used for the first Skip Level Meeting only.

Thank you for your contributions to GitLab

The Senior Manager will use this opportunity to say thank you to the team member for their specific contributions to GitLab. There is some preparation required for this. Here are some ideas for how to prepare:

- Ask the Team Members Manager for a short list of accomplishments for the team member
- Throughout the year keep track of all team members significant accomplishments in a separate document / spreadsheet
- Throughout the year keep track of thank you's from the Thank You channel or retrospectives for team members in a separate document / spreadsheet

About You

- What is keeping you from being more successful than you already are?
- What are your goals for the next 6 to 12 months?

About the GitLab Leadership (Sr. Manager and above)

- What do you need from the leadership team
- How would you change things if you were in my shoes?

About your Manager

- What areas of your career would you like your manager to pay more attention to?
- What areas of your team's processes, procedures would you like your manager to pay more attention to?

About your team

- What would you do differently if you were the manager of your team?
- What is the biggest challenge facing your team?

About the Company

- If there was one thing that you could "fix" in the company, what would it be?
- What is the biggest challenge facing your role in the organization?
- Do you know how your team's goals support the company's?

Additional Questions

- Is there anything else that we should have discussed?

Standard Skip Level Template

The template below is used after the first Skip Level Meeting has taken place.

Thank you for your contributions to GitLab

The Senior Manager will use this opportuntity to say thank you to the team member for their specific contributions to GitLab. There is some preparation required for this. Here are some ideas for how to prepare:

- Ask the Team Members Manager for a short list of accomplishments for the team member
- Throughout the year keep track of all team members significant accomplishments in a separate document / spreadsheet
- Throughout the year keep track of thank you's from the Thank You channel or retrospectives for team members in a separate document / spreadsheet

Your Team

- What's one thing your team should stop doing? Why did you choose that?
- How is your / your team's workload? Do you feel it's too little, too much, or the right amount?
- What do you think your team does really well?
- Can you describe the last major project you worked on? What's one thing that could be improved?

Giving Praise to your team

- Has anyone gone well above and beyond lately? What did they do?
- Who on your team makes those around them better? How do they do it?
- Is there a recent example would you like to share with me where you feel like your manager did a great job?

Your Manager

- Do you feel you're getting enough feedback from your manager? Why/Why not?
- What areas of your career would you like your manager to pay more attention to?
- What areas of your team's processes, procedures would you like your manager to pay more attention to?

For the Senior Manager

- Do you have any questions for me?
- Do you find these meetings useful? If yes, why? If not, what can I do to improve?

Frequency

The Senior Manager conducts approximately four Skip Level meetings a month. The Senior Manager will likely meet with everyone in their reporting structure a minimum of once a year. In many cases the Senior Manager will meet with team members twice a year.

What happens after the meeting

The Skip Level meeting is shared with your manager and your manager will then directly begin to address the concerns that were raised during the meeting.

SECTION: engineering/devops/dev/create/engineers/training.md

What training do we recommend for Engineers?

CREDIT Values

At GitLab we see CREDIT everywhere, it is in our 360 Feedback, Annual and Mid-Year Performance Reviews and in Promotion Documents. It is important to have evidence for this CREDIT but the training below provides opportunities to learn more about how team members can live our CREDIT values.

GitLab Value	Online Course(s)
Collaboration	Course(s) from the Building Trust and Collaborating with Others Pathway
Results	Enhancing your Productivity
Efficiency	Efficient Time Management
Diversity	Diversity, Inclusion & Belonging
Iteration	Interview about Iteration
Transparency	Communicating with Transparency

Working Remotely

We are experts in working remotely, so our handbook is the best resource for this area.

Remote Work Foundations (Handbook)

Feedback

Giving and Receiving feedback is challenging. The training below provides some best practices and ideas regarding how to handle feedback.

Giving & Receiving Feedback

Mentoring

Mentoring sounds easy however there can be pitfalls. The training below is a good resource to ensuring you engage in a productive Mentoring relationship.

Grow your Impact as a Mentor
Mentoring (Handbook)

Learning GitLab

For those looking to learn more about GitLab especially outside of the area where you work, the resources below are a great start.

Learning GitLab
Continuous Delivery with GitLab

SECTION: engineering/devops/dev/create/import/_index.md

About

The Import group is a part of the Foundations Stage.
The group supports the product by migrating between GitLab instances and from other providers.

This page covers processes and information specific to the Import group. See also the group direction page and the features we support per category.

How to reach us

To get in touch with the Import group, it's best to create an
issue in the relevant project (typically GitLab) and add the
~"group::import" label, along with any other appropriate labels. Then,
feel free to ping the relevant Product Manager and/or Engineering Manager.

For more urgent items, feel free to use the Slack Channel (internal): gimport.

Note that while we own the foundations of GitLab's APIs, the behaviour of most individual API endpoints is owned by
other teams. Please check the feature categorization page to ensure your
query is being directed to the correct group.

Team Members

The following people are permanent members of the group:

{{< engineering/stable-counterparts role="Foundations:Import and Integrate" >}}

Metrics

Our Engineering Metrics Dashboards can be found here.

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="import and integrate" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="import and integrate" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="import and integrate" >}}
{{< /tableau >}}

Work

The Product Manager uses milestone priority labels and compiles the list of Deliverable and Stretch issues following

the product prioritization process,
with input from the team, Engineering Managers, and other stakeholders.
The iteration cycle lasts from the 18th of one month until the 17th of the next,
and is identified by the GitLab version set to be released.

Issue Development Workflow

In general, we use the standard GitLab engineering workflow.

The easiest way for Engineering Managers, Product Managers, and other stakeholders
to get a high-level overview of the status of all issues in the current milestone,
or all issues assigned to a specific person, is through the Current milestone board,
which has columns for each of the workflow labels.

As owners of the issues assigned to them, engineers are expected to keep the
workflow labels on their issues up to date, either by manually assigning the new
label, or by dragging the issue from one column on the board to the next.

Once an engineer starts working an issue, they mark it with the `workflow::"in
dev"` label as the starting point and continue updating the issue throughout development.
The process primarily follows the guideline:

```
mermaid
graph LR

  classDef workflowLabel fill:428BCA,color:fff;

  A(workflow::in dev):::workflowLabel
  B(workflow::in review):::workflowLabel
  C(workflow::verification):::workflowLabel
  F(workflow::complete):::workflowLabel

  A -- Push an MR --> B
  B -- Merged --> C
  C --> D{Works on production?}
  D -- YES --> F
  F --> CLOSE
  D -- NO --> E[New MR]
  E --> A
```

If someone starts working on an issue but it has the same workflow label for a
week, the assignee has to leave a comment explaining the status of the issue.
We should write at least one comment every week that the issue is not moving.

Issue Boards

The work for the Import group can be tracked on the following issue boards:

- Current milestone board

Issue Labels

To maintain good label hygiene, please apply the correct labels when creating or triaging issues.

All issues should have:

- All of our section, stage and group labels:
  - ~"section::core platform"
  - ~"devops::foundations"
  - ~"group::import"
- One or more of the category labels:
  - ~"Category:API"
  - ~"Category:Importers"
  - ~"Category:Integrations"
  - ~"Category:Internationalization"
  - ~"Category:Webhooks"
- A type label
- A workflow label
- ~"backend" or ~"frontend" if appropriate

For issues related to importers, also apply an Importer: label. For example: ~"Importer:GitHub" or ~"Importer:Direct Transfer".

For issues related to integrations, also apply a scoped Integration:: label. For example: ~"Integration::Slack" or ~"Integration::Jira".

For issues related to our APIs, also apply either ~"api" for REST or ~"GraphQL" for GraphQL.

Once you have completed an issue and closed it make sure to add ~"workflow::complete".

Team members might find it helpful to use a comment template to help apply labels correctly. See an example here.

Capacity Planning

We use a lightweight system of issue weighting to help with capacity planning. These weights help us ensure that the amount of scheduled work in a cycle is reasonable, both for the team as a whole and for each individual. The "weight budget" for a given cycle is determined based on the team's recent output, as well as the upcoming availability of each engineer.

Since things take longer than you think,
it's OK if an issue takes longer than the weight indicates. The weights are intended to be used in aggregate, and what takes one person a day might take another person a week, depending on their level of background knowledge about the issue. That's explicitly OK and expected. We should strive to be accurate, but understand that they are estimates! Change the weight if it is not accurate or if the issue becomes harder than originally expected. Leave a comment indicating why the weight was changed and tag your EM so that we can

better understand weighting and continue to improve.

Weights

The weights we use are:

Weight	Description
1: Trivial	The problem is very well understood, no extra investigation is required, the exact solution is already known and just needs to be implemented, no surprises are expected, and no coordination with other teams or people is required. Examples are documentation updates, simple regressions, and other bugs that have already been investigated and discussed and can be fixed with a few lines of code, or technical debt that we know exactly how to address, but just haven't found time for yet.
2: Small	The problem is well understood and a solution is outlined, but a little bit of extra investigation will probably still be required to realize the solution. Few surprises are expected, if any, and no coordination with other teams or people is required. Examples are simple features, like a new API endpoint to expose existing data or functionality, or regular bugs or performance issues where some investigation has already taken place.
3: Medium	Features that are well understood and relatively straightforward. A solution will be outlined, and most edge cases will be considered, but some extra investigation will be required to realize the solution. Some surprises are expected, and coordination with other teams or people may be required. Bugs that are relatively poorly understood and may not yet have a suggested solution. Significant investigation will definitely be required, but the expectation is that once the problem is found, a solution should be relatively straightforward. Examples are regular features, potentially with a backend and frontend component, or most bugs or performance issues.
5: Large	Features that are well understood, but known to be hard. A solution will be outlined, and major edge cases will be considered, but extra investigation will definitely be required to realize the solution. Many surprises are expected, and coordination with other teams or people is likely required. Bugs that are very poorly understood, and will not have a suggested solution. Significant investigation will be required, and once the problem is found, a solution may not be straightforward. Examples are large features with a backend and frontend component, or bugs or performance issues that have seen some initial investigation but have not yet been reproduced or otherwise "figured out".

Anything larger than 5 should be broken down if possible.

Weights should account for both development and review time.

Security issues are typically weighted one level higher than they would normally appear from the table above. This is to account for the extra rigor of the patch release process.
In particular, the fix usually needs more-careful consideration, and must also be backported across several releases.

Backlog Refinement

Every week the engineering team completes a backlog refinement process to review upcoming issues. The goal of this effort is for all issues to have a

weight so we can more accurately plan each milestone using the estimated capacity for the team and the estimated issue weights.

In addition to this backlog refinement process, engineers on the team can add weights to any issues that are straight-forward and do not need backlog refinement.

This process happens in three steps.

Step 1: Identifying Issues for Refinement

The engineering manager will identify issues that need to be refined. On average we will try to refine up to 6 backend and up to 3 frontend issues per week. If there are issues that are good candidates for the backlog refinement process, please let the engineering manager know in the issue.

When picking issues to refine, we try to have themed refinements to reduce the context switching while the issues are being investigated. Here are some places to look:

- Infradev Issues
- Issues scheduled for the next milestone without weight
- Security Issues
- Performance issues
- Application Limits
- Missed-SLO
- Approaching-SLO
- Bugs
- Issues without weight

Once identified, the engineering manager will apply the ready for next refinement label, which will indicate the issues are ready for refinement.

The engineering manager will use the Refinement Bot to generate an issue with all the issues that have been identified for refinement.

Step 2: Refining Issues

Over the week, each engineer on the team will look at the list of issues selected for backlog refinement. Current backlog refinement issues.

For each issue, each team member will review the issues and provide the following information:

- Estimated weight.
- How to break down the issue into different issues or merge requests.

Some considerations:

- Keep the conversation on the original issues.
- During this process, the issue description and labels should be updated as more information is gathered.
- Does the issue need a feature flag?
- For efficiency, engineers can also skip the refinement of some issues depending on the feedback that we already have.
- Where the fix is clear and easy, we can assign the issue to ourselves, give it a weight of 1 and push the fix.

Step 3: Finalizing Refinement

After engineers have had a chance to provide input, the engineering manager will then:

- Apply a final weight. This could be the average of weights provided by the engineers, but the final decision is up to the engineering manager taking into consideration the uncertainty.
- Inform stable counterparts if there are any testing or security concerns.
- Remove the ready for next refinement label.
- Apply the right workflow:: label based on the outcome of the refinement. Example workflow::ready for development.

For any issues that were not discussed and given a weight, the engineering manager will work with the engineers to see if we need to get more information from PM or UX.

Requesting help

If you are not part of the Support organization, we recommend reaching out to them first, as they have greater availability and can assist with most common issues. There's a dedicated Slack channel sptpodimportandintegrate you can join, follow and ask questions in. However, there are times when in-depth technical knowledge is needed to resolve a customer issue, requiring the involvement of an engineer from the team.

Before requesting help from the Engineering team, please first review the GitLab documentation for the topic of your interest and the additional resources listed below:

- Importer Runbook
- GitLab Log Analysis Tool
- Jira playlist

If you cannot find the answer to your question in the resources listed above, please open a Request for Help (RFH) issue and use the SupportRequestTemplate-Import-Integrate template. Please ensure that you provide all the required information before reaching out to the team; otherwise, we will be unable to proceed with your request. New issues will be prioritized according to our internal triage process. Please note that we can only support requests for issues affecting the current and the two most recent minor GitLab versions (N-2). We cannot offer a fix for older versions. This is aligned with our maintenance policy for backports.

Milestone Doctors

In FY2025, on average 4-5 Request for Help (RFH) issues per month have been opened for feature categories that are owned by our team. Most of these issues are high-priority requests that involve the Engineering team to help resolve blocking issues for our customers. This type of ad-hoc work causes a lot of interruption while working on milestone Deliverables. To ensure these RFH issues are processed as quickly as possible by the Engineering team and to reduce context-switching time within the team, two engineers take on the "Milestone Doctor" role at every milestone. Their capacity for Deliverable work is reduced to 60% to allow taking over additional responsibilities as "Milestone Doctors".

Responsibilities

- Engage with Support and PS on new RFH issues
- Follow-up on long-lasting open issues
- Assist the Support team on customer calls
- Maintain team runbook documentation on how Milestone Doctors have successfully diagnosed problems
- Respond to questions in our team Slack channel gmanageimportandintegrate
- Update our FAQ with any learnings from the shift

Rotation schedule

- Each milestone two Backend Engineers take the role of a Milestone Doctor.
- Engineers claim shifts themselves on the Milestone Doctor schedule spreadsheet.
- At the beginning of a new milestone, Milestone Doctors update assignee section of the RFH template with their usernames.
- Given the current team size, every Backend Engineer is expected to sign up as Milestone Doctor once per quarter.
- At the end of each quarter, the EM assigns unassigned shifts for the upcoming quarter to engineers.

Working with Security

The group has an existing threat model to assist in identifying issues that may have security implications, but there are other considerations.

An Application Security Review should be requested when the issue or MR might have security implications. These include, but aren't limited to, issues or MRs which:

- falls under the threat model
- handles binary files (downloading, decompressing, extracting, moving, deleting)
- modifies or uses file manipulation services
- uses methods from Import/Export CommandLineUtil

Longer lived feature flags

This is a supplement to GitLab's common development guidance
for use of feature flags. It applies to all flag types besides the ops type.

Changes to Import features often happen in high-traffic code paths and have
led to outages on GitLab.com in the past. Outages are often to do with resource contention that

can
be difficult to see ahead of time in code review or in QA testing.

- Large imports can trigger thousands of workers.
- Integrations and webhooks are executed millions of times a day.
- Contention problems sometimes do not surface immediately, and only when large customers trigger the new code path.

For this reason we should prefer to keep feature flags in the codebase for a longer period of time than normal.
During this time the flag is enabled by default but can still be disabled quickly in the event of an incident.

In the past, we were able to quickly mitigate several incidents by disabling the feature:

- 2023-09-21: Group import allows impersonation of users in CI pipelines
- 2023-10-30: GitLab.com is down
- 2024-01-30: Sidekiq Apdex SLO

For changes within importers, integrations or webhooks we should prefer to:

1. Roll out the flag with /chatops as normal.
1. QA the changes using large data to proactively flush out any problems at scale.
   For importers, see our runbook
   for tips.
1. When you come to release the feature, change the feature flag to be defaultenabled: true
   rather than to remove it. This is the
   optional release the feature with the flag
   step on the flag rollout issue.
1. At this point the feature is considered released within the milestone, and can be
   announced in the release post as it will ship to self-managed customers.
1. Wait between 1-3 weeks where the flag exists in the codebase but remains enabled by default.
   Use the longer period for changes in areas of more contention, or if you feel it may take
   longer to detect problems for any reason.
1. After that period, remove the feature flag to complete the flag rollout process.

During a release

- When an issue is introduced into a release after Kickoff, an equal amount of weight must be removed to account for the unplanned work.
- Development should not begin on an issue before it's been estimated and given a weight.
- By the 15th, engineering merge requests should be merged. In other words, we assume code merged after the 15th will not be in the release. That allows time for the release to be finalized, and any associated release posts to be merged by the 17th. (This is an experiment starting with 13.11.)

Release posts

For issues which need to be announced in more detail, a release post can be automatically created using the issue.

When working on an issue, either in planning, or during design and development, you can use the release post item generator
to have the release post created and notify all the relevant people.

If you do not want an issue to have a release post, make sure that the issue does not have a release notes section or do not use a release post item:: label.

Proof-of-concept MRs

We strongly believe in Iteration and delivering value in small increments. Iteration can be hard, especially when you lack product context or are working on a particularly risky/complex part of the codebase. If you are struggling to estimate an issue or determine whether it is feasible, it may be appropriate to first create a proof-of-concept MR. The goal of a proof-of-concept MR is to remove any major assumptions during planning and provide early feedback, therefore reducing risk from any future implementation.

- Create an MR, prefixed with PoC:.
- Explain what problem the PoC MR is trying to solve for in the MR description.
- Timebox it. Can you determine feasibility or a plan in less than 2-3 days?
- Identify a reviewer to provide feedback at the end of this period.
- Close the MR. Provide a summary in the original issue on what you learned from the PoC, including product and performance implications.
  - State whether you are able to move forwards with implementation or not.
  - Please do not close the issue.

The need for a proof-of-concept MR may signal that parts of our codebase or product have become overly complex. It's always worth discussing the MR as part of the retrospective so we can discuss how to avoid this step in the future.

Retrospectives

We have 1 regularly scheduled "Per Milestone" retrospective, and can have ad-hoc "Per Project" retrospectives.

Per Milestone

The Import group conducts milestone retrospectives in GitLab issues. These include the engineers, UX, PM, and
all stable counterparts who have worked with that team during the milestone.

Participation by our team members is highly encouraged for every milestone.

These are confidential during the initial discussion, then made public in time
for each month's GitLab retrospective. For more information, see group retrospectives.

Per Project

If a particular issue, feature, or other sort of project turns into a
particularly useful learning experience, we may hold a synchronous or
asynchronous retrospective to learn from it. If you feel like something you're

working on deserves a retrospective:

1. Create an issue explaining why you want to have a retrospective and indicate whether this should be synchronous or asynchronous.
1. Include your EM and anyone else who should be involved (PM, counterparts, etc).
1. Coordinate a synchronous meeting if applicable.

All feedback from the retrospective should ultimately end up in the issue for reference purposes.

Tech Leads

Our group works with tech leads to help organize work on different topics and identify DRIs for them.

Characteristics of a Tech Lead

A tech lead is:

- an individual contributor with additional responsibilities. Every engineer regardless of their seniority is qualified to be a tech lead.
- a temporary role that is tied to a specific topic/project. We allow the team to have multiple tech leads at the same time for different topics/projects.
- not a manager.
- not an additional seniority level.

The Tech Lead role provides growth opportunity for engineers who are interested in adopting leadership skills.

Responsibilities of a Tech Lead

Tech leads wear many hats. Their responsibilities may differ from project to project but may include:

- Technical Vision and Architecture - Defining and evolving the overall technical architecture for a given project
- Technical Guidance - Providing technical guidance and mentoring to other developers on the team
- Planning and Prioritizing Work - Organizing the work by breaking down bigger tasks into smaller actionable items
- Tracking Progress - Tracking progress on commitments and reporting status updates
- Risk Management - Identifying, assessing and managing technical risks that may impact deliverables
- Coordination - Overseeing the work of others and helping remove blockers
- Technical documentation - Maintaining documentation of the technical architecture and code structure for other developers

Current Tech Leads

Below is an overview of topics that are overseen by a tech lead:

Topic	Tech Lead	Topic Link	Notes
Direct Transfer - User contribution mapping	Rodrigo Tomonari	Epic	-
Improve the efficiency of developer contributions to the importers	James Nutt	OKR	-
Congregate	tbd	https://gitlab.com/gitlab-org/gitlab/-/issues/428657	
GitHub Actions	tbd	https://gitlab.com/gitlab-org/manage/general-discussion/-/issues/17652	

Merge request roulette reviews

When areas of the Import codebase are changed, the reviewer roulette
will recommend that the merge request is reviewed by an Import team member. This will only happen when the merge request is
authored by people outside of the Import team. See this example of how the review recommendation looks.

The reasoning behind these special recommendations
is that other groups have some ownership of certain integrations or webhooks. Reviewing changes made by non-team members allows us to act as owners of foundational code and maintain a better quality of the Import codebase.

How roulette matches work

File paths of changes in a merge request are matched against a
list of regular expressions.
The roulette uses these hash values to recommend reviewer groups. For example, :importintegratebe and
:importandintegratefe will recommend Import backend and frontend reviews respectively. As the regex matches
are first match wins
and not cumulative, any other relevant reviewer groups like :backend or :frontend must also be included
in each hash value.

The regex list should be updated to match integrations or webhooks code whenever needed. The list matches our
commonly namespaced files, so new code in existing namespaces will always match.

To see which files in the GitLab repository produce a match, paste the following in a Rails console:

```
ruby
require Rails.root.join('tooling/danger/projecthelper.rb')

ALLFILES = Dir.glob('/');

def categoryregexs(category)
  matchingcategories = Tooling::Danger::ProjectHelper::CATEGORIES.select do |regexs, categories|
```

```
    next if regexs.isa?(Array)

    Array.wrap(categories).include?(category)
  end

  regexes = matchingcategories.map(&:first)
  Regexp.union(regexes)
end

def printfiles(category)
  regex = categoryregexs(category)

  puts ALLFILES.grep(regex).reject { |path| File.directory?(path) }.sort
end

puts "Backend:\n"
printfiles(:importintegratebe)

puts "Frontend:\n"
printfiles(:importintegratefe)
```

Monitoring

This is a collection of links for monitoring our features.

Grafana dashboards

- Import group dashboard which contain:
  - Links to various Kibana logs, filtered to our feature categories
  - Our error budget spend attribution
- Worker queues where you can switch queues with the queue dropdown

Sentry errors

- Matching "IntegrationsController"
- Matching "Integrations"
- Matching "Jira"

Kibana dashboards

See a list of all Import Kibana dashboards&sort=title&sortdir=asc).

Importer dashboards:

- Project Import/Export
- GitHub Import - Overview
- GitHub Import - Project import debug
- GitLab Direct Transfer
- User contributions mapping

API/Webhooks dashboards:

- REST and GraphQL API
- Webhooks

Kibana logs

GitLab for Jira Cloud app workers:

- JiraConnect::SyncMergeRequestWorker errors.
- JiraConnect::SyncBranchWorker errors.
- JiraConnect::SyncProjectWorker errors.
- All JiraConnect sync worker timeout errors.

Error budgets

GitLab uses error budgets to measure the availability and performance of our features.
Each engineering group has its own budget spend. The current 28-day spend for the Import team shows in this Grafana dashboard.

Error budget spend happens when either of the following exceeds a certain threshold:

- Error rate of an endpoint or worker
- Apdex (latency) of an endpoint

Determine the highest-impact fixes

To determine the highest-priority problems in our Grafana dashboard:

1. Go to the Error budget panel.
1. Expand Budget spend attribution. The Budget failures panel is ordered by top failures.
1. In Failure log links, click the corresponding links.

Fixing the top offenders will have the biggest impact on the budget spend.

Further resources

Learn more about error budgets with these resources:

- Error budgets and how they are calculated
- What Apdex is and how it works
- Error budget in Grafana dashboards
- Feature categorization: our code is attributed to us by featurecategory: :api, featurecategory: :integrations, featurecategory: :internationalization, featurecategory: :importers, and featurecategory: :webhooks

Usage data dashboards

You can view data for feature usage in Tableau.

- Centralized Product Usage Metrics Dashboard can be used to observe any chosen metric. To see data for e.g. Microsoft Teams integration, on the left in Select Metric Level choose PI, in the Select Metrics to view first uncheck All and then search for microsoft and check metrics for Microsoft Teams integration you're interested in, see example. You can choose timeframe and Dimention Paremeter, e.g deployment type. Another example is data for GitHub importer or webhooks usage by deployment
- Integrations Usage Dashboard shows all usage of integrations. You can filter in or out (keep only or exclude) any specific integration on the right hand side.
- Importer Usage Dashboard. This is still work in progress.
- User Contribution Mapping Usage Dashboard shows data on placeholder users created during imports.

Links and resources {links}

{{% include "includes/engineering/foundations/shared-links.md" %}}

- Milestone retrospectives
- Our Slack channels
  - Manage:Import gmanageimportandintegrate
  - Daily standups gmanageimportandintegratedaily
- Issue boards
  - Current milestone board
- Contribution guides
  - Principles of importer design
  - Contributing to Direct Transfer
    - Feedback issue
- Onboarding videos (GitLab Unfiltered Youtube)
  - Direct Transfer (formerly known as GitLab Migration)
  - Introduction to GitHub Importer
  - File based GitLab Import/Export
  - Remote S3 Import Example

SECTION: engineering/devops/dev/create/remote-development/_index.md

Overview

The group is part of Create Stage in the Dev Sub-department. We focus on two categories: Workspace and the Web IDE.

· Group Principles

Create:Remote Development Principles: What Are the Create:Remote Development Group Principles?

· Team Members

The following people are permanent members of the Remote Development Engineering Group:

Engineering Manager & Engineers

{{< team-by-manager-slug "adebayoa" >}}

Product, Design, Technical Writing, Security & Quality

{{< engineering/stable-counterparts role="(Product Manager|Technical Writer|Software Engineer in Test|Security Engineer).(Create:Remote Development|Create \(Remote Development)|Dev\:Create" >}}

· Category DRIs

Category	DRI
Workspaces	{{< member-by-name "Vishal Tak" >}}
Web IDE	{{< member-by-name "Enrique Alcántara" >}}

· Architecture Design Document

Design documents are the primary artifact that the architecture design workflow revolves around. A design document describes a technical vision and a set of principles that will guide feature implementation, as we move forward. It acts as guardrails to keep team aligned.

- Workspaces

· New Hires

As the Remote Development team and tech stack continue to mature, it's essential to have a team-specific onboarding process for new hires. This checklist is designed to guide new team members through the key areas and processes specific to our team, starting two weeks after company onboarding. It covers our mission, essential tools, and workflows related to the Web IDE and Workspaces. Existing team members are encouraged to review the checklist regularly and contribute any missing or updated information to ensure it remains accurate and useful for newcomers. You can find the template https://gitlab.com/gitlab-com/create-stage/remote-development/-/blob/main/.gitlab/issuetemplates/remote-development-onboarding.md.

·· How to reach us

Depending on the context here are the most appropriate ways to reach out to the Remote Development Group:

- Slack Channel: gcreateremotedevelopment
- Slack Groups: @create-remote-development-team (entire team) and @create-remote-development-engs (just engineers)

·· Capturing Customer Engagements

To improve our understanding and traceability of customer needs and to ensure followups action

items are systematically done, we want to capture customer engagement notes in a SSoT. Please use the confidential issues below to capture all customer engagements for the feature categories:

- Web IDE Customer Engagements
- Workspaces Customer Engagements

These epics are meant for internal team members